

detailed commit messages will save you a lot of grief in determining exactly how far back you need to go.

If you forget to add the `-m "<message>"` command line argument, Git will open the Vim text editor in the Terminal window. Vim is beyond the scope of this book, so press the ESC key and enter `qa!` to safely exit Vim and cancel the commit. Then enter the `git commit` command again, this time with the `-m "<message>"` command line argument.

For examples of what professional commit messages look like, check out the commit history for the Django web framework at <https://github.com/django/django/commits/master>. Because Django is a large, open source project, the commits occur frequently and are formal commit messages. Infrequent commits with vague commit messages might work well enough for your small, personal programming projects, but Django has more than 1,000 contributors. Poor commit messages from any of them becomes a problem for all of them.

The files are now safely committed to the Git repo. Run `git status` one more time to view their status:

```
C:\Users\Al\wizcoin>git status
On branch master
nothing to commit, working tree clean
```

By committing the staged files, you've moved them back to the committed state, and Git tells us that the working tree is clean; in other words, there are no modified or staged files. To recap, when we added files to the Git repo, the files went from untracked to staged and then to committed. The files are now ready for future modifications.

Note that you can't commit folders to a Git repo. Git automatically includes folders in the repo when a file in them is committed, but you can't commit an empty folder.

If you made a typo in the most recent commit message, you can rewrite it using the `git commit --amend -m "<new commit message>"` command.

Using git diff to View Changes Before Committing